Project Member and Task Management Module Design

Student Project Planning and Progress Tracking System

Round 2 integration revision; targeted consistency repair 26 October 2026; DRAFT PENDING TEAM REVIEW

F1 — Project and Member Management

F2 — Task Management

Group 9 — Xingmang

Original contribution: Zhang Weiyi. Round 2 editorial integration prepared with Codex assistance for Wei Mingrui; not author approval.

Source week 2026-10-22 | Round 2 integration revision 2026-10-25 | DRAFT PENDING TEAM REVIEW

Document control	
Status	DRAFT PENDING TEAM REVIEW; design protocol only, no implementation or executed software tests
Scope	F1 Project and Member Management; F2 Task Management
Inputs	URS (10 Oct 2026, 24 pp); Requirements Analysis (10 Oct 2026, 36 pp); Proposal (8 pp); Chapter 6 Structured Software Design
Out of scope	Application code, UI design, database schema, API framework, and the internal design of F3– F6
Author / owner	Zhang Weiyi (Group 9)
Reviewers	Intended team reviewers: Wei Mingrui and Zhang Yuyang; no review approval is recorded here.

Editorial provenance: the advertised source DOCX and draw.io were not supplied. This ROUND2 report reconstructs the received PDF as editable text/tables, corrects the identified integration conflicts and adds explicit shared contracts. Source authorship remains Zhang Weiyi; no original-author approval is claimed.

1 Executive Summary

This document defines a preliminary general design for the project and task management subsystem (F1 and F2). It converts the verified revised URS and requirements-analysis models into a reviewable module structure with explicit responsibilities, interfaces, rules and validation scenarios. It does not prescribe an implementation technology and does not contain application code.

Design at a glance

A thin Management Facade coordinates the original seven public write entries and an explicit deleteTask contract for the already-described task-lifecycle operation. Four cohesive modules own project data, membership/roles, task records and progress notes. MG-M2 supplies one permission policy used through shared coordination. Tasks store only NOT_STARTED / IN_PROGRESS / COMPLETED; F5 alone derives Blocked from task status and valid F3 links.

Key design decisions

- Roles belong to a Project Membership, not to a global user account. The same student may be Leader in one project and Ordinary Member in another.
- The facade checks membership and edit permission before every write; branch modules do not trust the UI and do not duplicate permission rules.
- Member departure ends active membership and clears task assignments, but never deletes tasks, dependencies or notes.
- All writes are transactional. Validation and permission checks complete before any record changes; a failed save returns an error and preserves the prior valid data.
- F3/F4/F5/F6 receive stable task facts and change notifications; F1/F2 do not duplicate dependency, schedule, blocked-warning or dashboard logic.

The design retains nine open decisions from the source: invitation delivery, leader succession, exact project fields, task field/assignment rights, leader agreement, task deletion/reference effects, note history/editing, status/early-completion semantics, and ordinary-member project-description editing. Section 9 preserves these decisions. Round 2 shared interface protocols are design proposals, not customer approval.

2 Scope Sources and Design Approach

2 1 Scope

In scope	Out of scope
Create and edit project information; planned start, submission deadline and internal targets.	Dependency recording/cycle detection (F3), schedule calculation/critical activities (F4), dashboard and risk warnings (F5/F6).
Project membership, project-specific roles, member departure and permission decisions.	Authentication, invitation delivery/acceptance, email or notification mechanisms.
Task creation, editing, assignment, permitted deletion and status updates.	Task dependency graph storage, automatic schedule recalculation, effort tracking or resource levelling.
Short task progress notes and their project-member-only viewing.	Comment threads, instant messaging, file attachments, export and undo.
Conceptual interfaces, invariants, failures and validation scenarios.	UI layout, database tables, API framework, deployment and source code.

The design is limited to F1/F2. F3–F6 are consumers of stable task and membership facts, not submodules of this design. Where this document mentions them, it defines only the information boundary.

2 2 Sources Read

Source	Use in this design
Student_Project_Planning_and_Progress_Tracking_System_URS_REVISED.pdf (24 pp, 10 Oct 2026)	FR-01–FR-06, NFR-01/04/05, roles, data handling, error handling, open questions OQ-05/OQ-06/OQ-08.

Source	Use in this design
Student_Project_Planning_and_Progress_Tracking_System_Requirements_Analysis_REVISED.pdf (36 pp, 10 Oct 2026)	Actors, use cases UC-01/02/03/07/08/09/15, domain objects Project, ProjectMembership, Task, ProgressNote, and sequence behaviour.

Source	Use in this design
Student_Project_Planning_and_Progress_Tracking_System_Proposal(1).pdf (8 pp)	Feature scope F1/F2, project/task concepts, positive calendar-day durations, member removal expectation.
SE_Chapter6-Structured Software Design-2026.10.22.pdf (105 slides)	General design, modularity, cohesion/coupling, information hiding, hierarchy/structure charts and the seven heuristic rules.
Project repository docs (README, REQUIREMENTS.md, ARCHITECTURE.md, TEAM.md)	Module boundary: management owns F1–F2; scheduling and monitoring are separate; Task.status has three stored values.

2 3 Design Approach

The design applies Chapter 6 modular general-design principles to the existing object-oriented requirements analysis; it does not claim formal derivation from a DFD. The system is partitioned into modules with one clear reason to change, arranged in a three-level hierarchy. Each module hides one design decision: project date rules, project-specific permission rules, task-field/status rules, or note retention rules. Modules communicate through small data contracts (identifiers and value objects), not shared global state.

The hierarchy is deliberately shallow. The four functional modules have clear ownership; the explicit MG-M0 root is a control/coordination module rather than a fifth data owner. This follows the Chapter 6 guidance on moderate depth, high fan-out at the top, and simple interfaces, while avoiding over-decomposition.

2 4 Boundary with the Other Functional Modules

Module	Owns	F1/F2 provides	F1/F2 must not do
F3 Dependency Management	Validation, cycle rejection, maintenance and provision of valid dependency links. F5 alone derives Blocked.	Coherent task identities/project context, revision and committed SavedChange facts; F3 owns link records.	Store dependency edges or decide blocked status.
F4 Schedule Analysis	Project duration, task timing, critical activities, recalculation.	projectId, plannedStartDate, taskId, durationDays, revision and committed SavedChange facts. F1 also supplies projectDeadline for downstream date comparisons; it does not constrain CPM.	Calculate schedules or shorten duration because a task is completed.
F5 Progress Tracking	Completion counts, blocked/overdue/ongoing views, dashboard.	taskId, projectId, name, assigneeId (MemberId|null), status, taskDeadline and revision; optional note text through an authorized read.	F5 derives Blocked from valid F3 links and F2 raw status. Notes never alter completion counts; F5 does not write links or perform CPM.
F6 Schedule Risk Monitoring	Only retained FR-15 warning composition: unfinished overdue, blocked critical and project-deadline conflict. No CPM or finish-date calculation.	TaskSnapshot status, taskDeadline, assigneeId; ProjectSnapshot projectDeadline and revision; committed SavedChange facts.	F1/F2 does not compose warnings. F6 uses F5 facts and current available F4 results, without performing CPM or inventing a finish date.

Table 1 — Responsibility boundary. F1/F2 supplies facts; F3–F6 own their derived results.

3 Requirements Review

The following review keeps the URS identifiers unchanged. Each row states the design consequence, not a new requirement. FR-01–FR-06 are the functional scope of this document; NFR-01, NFR-04 and NFR-05 are the quality constraints that most directly shape the module contracts.

3 1 Functional Requirements

ID	Requirement summary	Design implication	Module
FR-01	Create a project with name, description, submission deadline, internal targets, member list and planned start.	A Project record owns descriptive fields and dates. Creation also establishes the creator's Leader membership. The agreed date policy is validated before commit; ordering and inclusion remain candidate rules.	MG-M1, MG-M2; MG-M0 coordinates
FR-02	Manage membership; when a member leaves, retain tasks and mark them unassigned.	Membership is a first-class record. Departure is a coordinated transaction: end active membership, clear assignee on that member's tasks, retain tasks/notes/dependencies. Invitation delivery is open.	MG-M2; MG-M3 for unassignment; MG-M0
FR-03	Create and edit task records: name, owner, description, estimated duration, deadline and status.	Task owns candidate fields and positive calendar-day duration. Assignment must reference a member of the same project. Field requirements and edit rights remain partly open.	MG-M3; MG-M2 for membership/permission
FR-04	Authorised members update task status centrally; a Blocked label must not prevent updates.	Status setter accepts only NOT_STARTED, IN_PROGRESS or COMPLETED. Blocked is rejected as a stored value and remains a derived warning. Status changes refresh progress consumers.	MG-M3; MG-M2 for permission

ID	Requirement summary	Design implication	Module
FR-05	Provide a short text progress note on a task.	ProgressNote is a separate record linked to Task. Adding a note does not change status or completion percentage. Length, history and editing rights are open.	MG-M4; MG-M3 for task identity; MG-M2 for permission
FR-06	Project Leader and Ordinary Member roles; leaders manage deletion, assignments, deadlines and roles; members create tasks and update status; restricted edits require leader agreement.	Role is stored on ProjectMembership. A single permission decision returns ALLOW / DENY / LEADER_AGREEMENT_REQUIRED. No approval workflow is designed; the decision is a policy result.	MG-M2; enforced by MG-M0, used by MG-M1/MG-M3/MG-M4

3 2 Non Functional Requirements Considered

ID	Constraint	Design response
NFR-01 Usability	Plain English labels, understandable status/blocker reasons, simple task-entry forms, no enterprise-style complexity.	Requests use a small field set; errors identify the field and a plain-language reason. Status labels are explicit text, not colour alone. Module contracts do not expose internal codes as the only explanation.
NFR-04 Security and Access Control	Project details, tasks, assignments and notes are visible only to members of that project; edit rights are separate from visibility.	Every operation is project-scoped and reauthorizes access. Unknown/inaccessible project references return INVALID_OR_INACCESSIBLE_PROJECT with no private payload; task references use INVALID_OR_INACCESSIBLE_TASK. Viewing permission remains separate from edit permission.
NFR-05 Reliability	Failed saves explain the problem, retain entered input and preserve valid saved data, including the previous valid plan after a rejected edit.	MG-M0 performs permission/validation first, then a single transaction across affected records. On failure it rolls back, returns safe field errors and input-retention information; no partial membership/task/note change is committed.

3 3 Requirement to Module Matrix

Requirement	MG-M0 Facade	MG-M1 Project	MG-M2 Member	MG-M3 Task	MG-M4 Notes
FR-01 Create project	Coordinates	Owns	Creates leader membership	—	—

Requirement	MG-M0 Facade	MG-M1 Project	MG-M2 Member	MG-M3 Task	MG-M4 Notes
FR-02 Manage members	Coordinates	Reads project context	Owns	Unassigns on departure	Retains notes
FR-03 Create/edit task	Coordinates	Provides project context	Checks membership/role	Owns	—
FR-04 Update status	Coordinates	—	Checks permission	Owns	—
FR-05 Notes	Coordinates	—	Checks permission	Provides task identity	Owns
FR-06 Permission control	Enforces decision	Uses for project edits	Owns policy	Uses for task edits	Uses for note edits
NFR-01 Usability	Simple result envelope	Plain date/field errors	Plain restriction messages	Plain task/status errors	Plain note errors
NFR-04 Security and Access Control	Enforces membership	Project-scoped reads	Owns membership facts	Project-scoped task reads	Private notes
NFR-05 Reliability	Transaction/ rollback	Atomic project save	Atomic membership change	Atomic task save	Atomic note save

Table 2 — Requirement ownership. “Coordinates” means sequencing and transaction control, not business-rule ownership.

4 Module Decomposition and Rationale

4 1 Preliminary Module Structure

Module	Subfunctions	Owns
MG-M0 Management Facade	Operation dispatch; transaction; permission check; change facts.	Use-case sequencing and consistency, not project/task data.
MG-M1 Project Management	Create Project; Edit Project Information; Manage Project Dates.	Project descriptive data and dates.
MG-M2 Membership and Permission Policy	Manage Membership; Manage Project-Specific Roles; Handle Member Departure; Check Permission.	Owns membership and permission facts only. MG-M0 asks MG-M3 for affected assignments and returns the combined DepartureResult.
MG-M3 Task Management	Create Task; Update Task; Assign Task; Delete Task; Update Task Status.	Task record fields, assignment and stored status.
MG-M4 Progress Notes	Add Short Task Note; View Task Notes.	Short note text and project-member-only note retrieval.

4 2 Why These Modules

The candidate subdivisions in the brief are retained, with three refinements. First, Check Permission is placed inside Membership and Permission Policy rather than as a fifth top-level module: a role is an attribute of project membership, and a separate module would create an artificial extra branch for one small policy decision. Second, Project Management owns only project fields and dates; it does not own the member list, which would duplicate MG-M2. Third, Progress Notes stays separate from Task Management because note text has different lifecycle questions (length, editing, history) and can be retrieved independently; keeping it separate prevents the task module from becoming a mixed task-and-comment module.

Task assignment is an operation inside Task Management, not a separate module: it changes the task's assignee and uses the same task record, so separating it would create unnecessary interface traffic. Task deletion remains a distinct operation because it has its own permission and reference-cleanup policy, but it is still part of the task lifecycle.

The root Management Facade is the control module. It sequences multi-record operations (project + leader membership; departure + task unassignment; task deletion + note/dependency reference policy) and owns the transaction boundary. It does not contain project, membership or task rules.

4 3 Cohesion and Coupling

Module	Cohesion	Coupling and boundary
MG-M0 Facade	Functional/control cohesion: every operation follows the same check–dispatch–commit sequence.	Depends on the four modules and a persistence port. It is the only module allowed to coordinate them; branch modules do not call each other directly.
MG-M1 Project Management	Functional cohesion around project attributes and date validation.	Receives projectId and project patches; returns ProjectView. It does not read task or membership tables. Creator membership is requested through MG-M0/MG-M2.
MG-M2 Membership and Permission Policy	Functional/communicational cohesion around membership, role and access decisions.	Owns membership and permission facts only. MG-M0 asks MG-M3 for affected assignments and returns the combined DepartureResult.
MG-M3 Task Management	Functional cohesion around one task record and its lifecycle.	Uses IDs and value objects; depends on MG-M2 only through the facade's permission decision. Dependency/note reference checks are ports supplied by MG-M0, not direct F3/MG-M4 calls.
MG-M4 Progress Notes	Functional cohesion around short note text and retrieval.	Depends on task identity and membership; does not change task status or completion counts. Notes are read only by project members.

The design uses data coupling (identifiers, value objects and result records) and limited control coupling through the facade's operation dispatch. There is no common/global mutable data and no content coupling: no module

reaches into another module's internal records. Interfaces are narrow and explicit, which keeps a change to date rules, role rules or note retention local to one module.

4 4 Structure Metrics

Measure	Value	Comment
Depth	3	Root → functional module → operation.
Root fan-out	4	Project, Member, Task, Notes; within the Chapter 6 average of 3–4.
Maximum branch fan-out	5	Task Management; within the suggested 5–8 upper bound but reviewed for simplicity.
Fan-in	MG-M0 is the main caller; MG-M2 is reused by all write operations through the facade.	Permission logic has high fan-in, which is desirable for consistency.
Module size	Leaf operations are small and single-purpose.	No operation mixes project, membership, task and note rules.
Interface style	IDs + small value objects + result/error records.	No shared database rows or global flags cross the boundary.

The structure follows the Chapter 6 heuristics: high cohesion/low coupling, moderate size, moderate depth and width, simple interfaces, a single entry and exit per operation, and predictable results for the same input. The action scope of a permission decision remains inside the facade's control scope; no module makes a permission decision that affects an unrelated branch.

5 Module Hierarchy

Figure 1 shows an explicit MG-M0 Management Facade above four data-owning branches MG-M1 to MG-M4. Check Permission is a policy subfunction of MG-M2. The lines express decomposition, not runtime messages, and all leaf operations retain their source responsibilities.

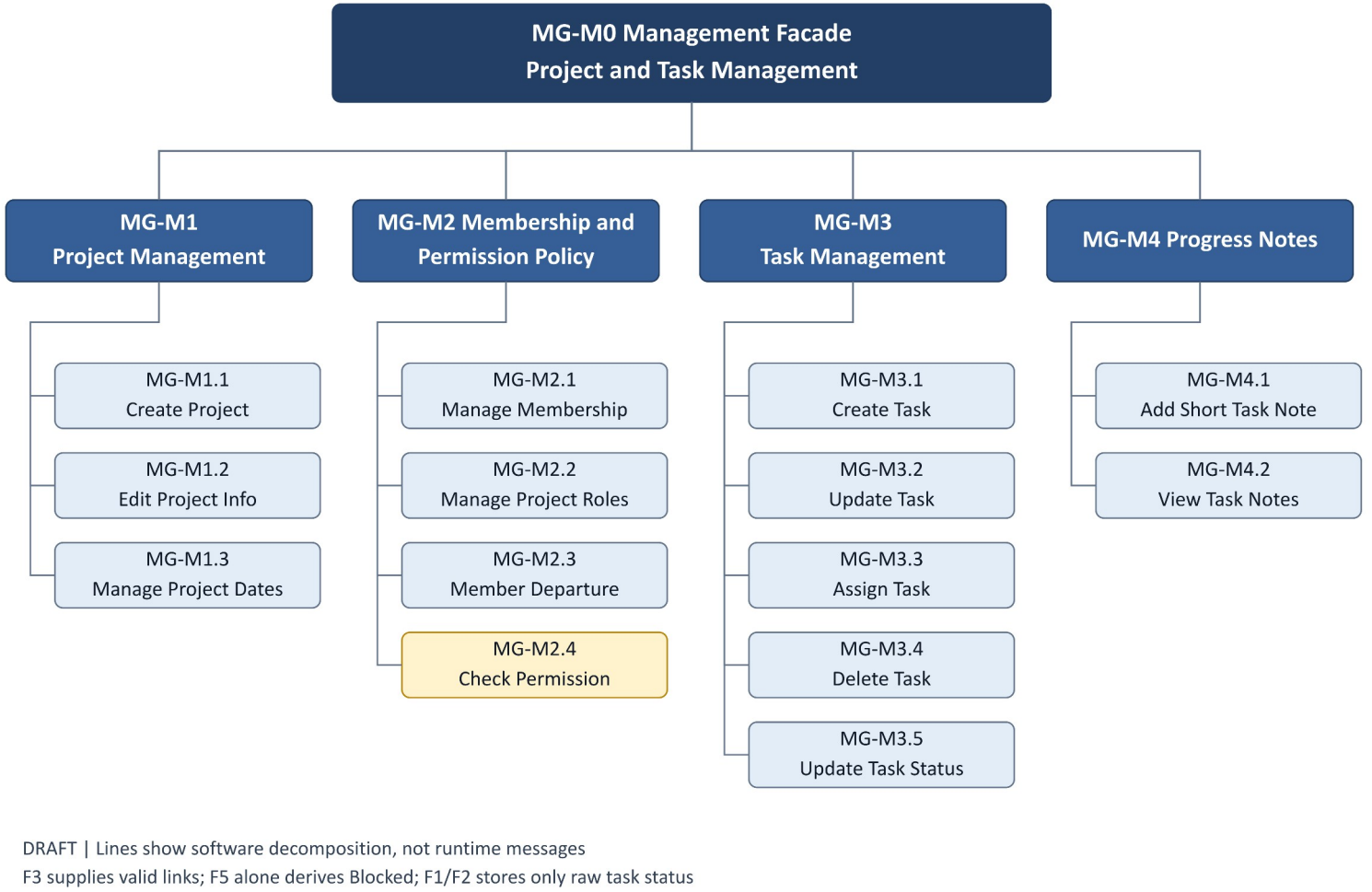


Figure 1. F1/F2 module hierarchy with explicit MG-M0 facade. Solid boxes are modules or operations; the permission-policy leaf remains under MG-M2. Editable source: 01_Project_Task_Hierarchy_ROUND2.drawio.

ID	Module / operation	Parent	Purpose
MG-M0	Management Facade	—	Sequence use cases, enforce permission, own transaction and change facts.
MG-M1	Project Management	MG-M0	Project attributes and dates.
MG-M1.1	Create Project	MG-M1	Create project shell and establish creator membership.
MG-M1.2	Edit Project Information	MG-M1	Edit name/description under permission.
MG-M1.3	Manage Project Dates	MG-M1	Maintain planned start, submission deadline and internal targets.
MG-M2	Membership and Permission Policy	MG-M0	Membership, project-specific roles, departure and permission decisions.
MG-M2.1	Manage Membership	MG-M2	Add/end active membership within the agreed policy.
MG-M2.2	Manage Project-Specific Roles	MG-M2	Assign/change Leader or Ordinary Member within one project.

ID	Module / operation	Parent	Purpose
MG-M2.3	Handle Member Departure	MG-M2	End active membership and provide membership-change facts; MG-M0 queries MG-M3 for affected assignments; tasks are retained.
MG-M2.4	Check Permission	MG-M2	Return ALLOW / DENY / LEADER_AGREEMENT_REQUIRED for an operation.
MG-M3	Task Management	MG-M0	Task lifecycle and stored status.
MG-M3.1	Create Task	MG-M3	Create a task in a project with valid fields and assignee.
MG-M3.2	Update Task	MG-M3	Edit task details, duration, deadline or owner under permission.
MG-M3.3	Assign Task	MG-M3	Assign to a project member or unassign.
MG-M3.4	Delete Task	MG-M3	Delete under leader permission and the agreed reference policy.
MG-M3.5	Update Task Status	MG-M3	Set NOT_STARTED / IN_PROGRESS / COMPLETED; reject Blocked as stored value.
MG-M4	Progress Notes	MG-M0	Short task notes and their read access.
MG-M4.1	Add Short Task Note	MG-M4	Create a short note linked to one task.
MG-M4.2	View Task Notes	MG-M4	Return notes to project members only.

6 Module Specifications

Each module is described by purpose, responsibilities, inputs, outputs, required information, dependencies, relevant requirements and error handling. The subfunction tables add the operation-level contract. The design is conceptual: “record” and “repository” do not prescribe a database or API.

6 1 Management Facade

Element	Description
Purpose	Provide one controlled entry point for the eight F1/F2 write operations and keep cross-module changes consistent.
Main responsibilities	Validate the request envelope; obtain a permission decision from MG-M2; invoke the owning module; manage the transaction; return a result and publish change facts.

Element	Description
Inputs	actorId, projectId (except createProject), operation name, operation payload.; actorId is UserId; expectedRevision for each existing-project mutation.
Outputs	Result envelope: success + view/identifier, or failure + code + plain message + field errors. Change facts such as ProjectChanged, MembershipChanged, TaskChanged, TaskStatusChanged, NoteAdded. The facade wraps successful writes in MutationResult<T> with revision and SavedChange.
Required information	Actor identity, project context, target record ID, requested operation, permission decision, transaction handle.
Dependencies	MG-M1–MG-M4; persistence/unit-of-work port; read-only reference ports for F3 dependency references and F4 schedule refresh.
URS requirements	FR-01–FR-06; NFR-04 (authorisation), NFR-05 (atomic save), NFR-01 (plain errors).
Error handling	Apply shared safe error mapping. Failure leaves source/revision unchanged and retains caller input; return no private payload. Do not dispatch after a failed permission/validation check. Roll back all writes on failure. Return safe field errors and retain submitted input; never expose partial project data to a non-member.

Subfunctions: request dispatch, permission gate, transaction/rollback, change-fact publication. The facade has no project, membership or task business rules of its own.

6 2 Project Management

Element	Description
Purpose	Own the project record and its dates; provide a stable project context for all other modules.
Main responsibilities	Create the project shell; edit descriptive information; validate and maintain planned start, submission deadline and internal target dates; return a ProjectView.
Inputs	ProjectDraft (name, description, plannedStartDate, projectDeadline, internalTargets); ProjectPatch; date changes; actor/project context.; actorId is UserId; expectedRevision for each existing-project mutation.
Outputs	ProjectView (projectId, name, description, dates, revision-tagged source values); ProjectChanged fact with changed field names. The facade wraps successful writes in MutationResult<T> with revision and SavedChange.
Required information	projectId, name, description, plannedStartDate, projectDeadline, internalTargets (representation open), creator/leader identity.
Dependencies	MG-M0 for permission/transaction; MG-M0 coordinates creator membership with MG-M2; persistence port; F4 receives date-change facts. It does not read task or note records.
URS requirements	FR-01; FR-06 (leader permission for project dates); NFR-04; NFR-05.

Element	Description
Error handling	Apply shared safe error mapping. Failure leaves source/revision unchanged and retains caller input; return no private payload. Missing/invalid name or dates: field errors, no write. Date-order rejection follows the later agreed policy; start/projectDeadline and internal-target ordering remain candidates. Unauthorised edit: deny and leave prior values. Save failure: rollback and retain submitted values for the caller.

Subfunction	Responsibility	Input → output	Main errors
Create Project	Validate fields; create project; add creator as Leader through MG-M2; commit atomically.	ProjectDraft + creator → ProjectView	Invalid field; date conflict; save failure
Edit Project Information	Edit name/description under the project-edit permission; do not change membership or tasks.	ProjectPatch → ProjectView	Not permitted; invalid text; save failure

Subfunction	Responsibility	Input → output	Main errors
Manage Project Dates	Maintain planned start, submission deadline and internal targets; validate ordering.	DatePatch → ProjectView + date-change fact	Invalid date; start after deadline; not permitted

Open Decision OD-03: exact required project fields, date-boundary rules and the representation of internal target dates are not fixed by the URS. The module keeps these as validation policy inputs rather than hard-coding a representation.

6 3 Membership and Permission Policy

Element	Description
Purpose	Own project membership and project-specific roles, handle departure safely, and provide the single permission decision used by all writes.
Main responsibilities	Add/end active membership; assign/change Leader or Ordinary Member; process departure; provide membership-change facts; MG-M0 queries MG-M3 for affected assignments; decide ALLOW / DENY / LEADER_AGREEMENT_REQUIRED for an operation.
Inputs	actorId (UserId), projectId, memberId for existing-membership actions, userId for additions, membership action, role, requested operation/resource for permission checks.; actorId is UserId; expectedRevision for each existing-project mutation.
Outputs	MembershipView; PermissionDecision; MembershipChanged fact. MG-M0 combines MG-M3 affected assignments into DepartureResult. The facade wraps successful writes in MutationResult<T> with revision and SavedChange.
Required information	memberId, userId, projectId, role, membership state, actor role, operation and target resource.
Dependencies	MG-M0 for transaction; MG-M3 is asked by MG-M0 to clear assignments after departure; persistence port. It does not own task status or notes.
URS requirements	FR-02; FR-06; NFR-04; NFR-05.
Error handling	Apply shared safe error mapping. Failure leaves source/revision unchanged and retains caller input; return no private payload. Target not found: not-found error. Actor not a member: deny without project data. Ordinary member requesting a restricted action: LEADER_AGREEMENT_REQUIRED. Last-leader/succession policy: Open Decision. Failed save: no membership or assignment change.

Subfunction	Responsibility	Input → output	Main errors
Manage Membership	Add a member record or end active membership under the agreed policy. Invitation delivery/acceptance is not designed.	MembershipCommand → MembershipView	Not permitted; duplicate membership; target not found; save failure
Manage Project-Specific Roles	Set Leader or Ordinary Member on one membership; never create a global role.	RoleChange → MembershipView	Not permitted; leader-succession policy; save failure
Handle Member Departure	End active access; retain historical member references; MG-M0 asks MG-M3 for affected assignments and clears them atomically. Do not delete tasks, notes or F3 links.	DepartureRequest → DepartureResult	Last-leader policy; not permitted; save failure
Check Permission	Evaluate project membership and role for an operation; return a decision, not an approval workflow.	PermissionQuery → PermissionDecision	Not a member; unknown operation

Permission decision semantics: ALLOW means the operation may proceed; DENY means the actor has no right; LEADER_AGREEMENT_REQUIRED means the ordinary member's request is not executed by this call. The facade may offer the leader the corresponding operation. Whether “agreement” is a separate approval or the leader performing the edit is Open Decision OD-05; no approval workflow is invented here.

Departure rule: MG-M2 ends active membership and does not inspect or delete tasks. MG-M0 obtains the affected assignments from MG-M3, coordinates assigneeId=null with the membership change in one atomic source commit, and returns a combined DepartureResult. Tasks, valid dependencies, notes, statuses and durations remain. Historical authorMemberId references remain stable after active access ends; their storage representation is deferred.

6 4 Task Management

Element	Description
Purpose	Own the task record and its lifecycle: creation, editing, assignment, permitted deletion and stored progress status.
Main responsibilities	Validate task fields and same-project assignment; maintain positive calendar-day duration; enforce status vocabulary; reject Blocked as stored value; return TaskView and change facts.
Inputs	projectId, taskId, TaskDraft/TaskPatch, assigneeId (nullable), durationDays, taskDeadline, requested status.; actorId is UserId; expectedRevision for each existing-project mutation.
Outputs	TaskView (taskId, projectId, name, description, assigneeId/unassigned, durationDays, taskDeadline, status); TaskChanged/TaskStatusChanged/TaskDeleted facts. The facade wraps successful writes in MutationResult<T> with revision and SavedChange.
Required information	taskId, projectId, name, description, assigneeId or unassigned, durationDays finite and positive when present; mandatory/precision policy pending, taskDeadline, status in {NOT_STARTED, IN_PROGRESS, COMPLETED}.
Dependencies	MG-M0 for permission/transaction; MG-M0 supplies MG-M2 permission decisions; read-only dependency-reference and note-reference ports supplied by MG-M0; persistence port.
URS requirements	FR-03; FR-04; FR-06; NFR-05. Status baseline follows the repository architecture: three stored values.
Error handling	Apply shared safe error mapping. Failure leaves source/revision unchanged and retains caller input; return no private payload. Invalid/blank name, non-positive duration, invalid deadline: field errors, no write. Assignee not a project member: reject. Blocked requested: INVALID_STATUS. Unauthorised: deny. Delete with unresolved references: policy-dependent result; never leave dangling references. Save failure: prior task unchanged.

Subfunction	Responsibility	Input → output	Main errors
Create Task	Create a task with proposed initial NOT_STARTED; validate assignee membership and agreed required fields. Initial-state/transition approval remains open.	TaskDraft → TaskView	Invalid field; assignee not member; not permitted; save failure
Update Task	Edit name/description/duration/deadline/owner under the permission policy; restricted edits return a decision.	TaskPatch → TaskView	Invalid field; not permitted; requires leader agreement; save failure
Assign Task	Assign to a project member or unassign; reassignment of others' work is restricted.	AssignmentChange → TaskView	Assignee not member; not permitted; requires leader agreement
Delete Task	Delete only under leader permission and the agreed reference policy.	taskId → TaskDeleted	Not permitted; unresolved dependency/note references; save failure
Update Task Status	Set one of the three stored states; Blocked is not accepted; blocked warning does not lock the update.	StatusChange → TaskView + status fact	Invalid status; not permitted; task not found; save failure

Status rule: TaskStatus is NOT_STARTED, IN_PROGRESS or COMPLETED. Blocked is an independent warning derived solely by F5 from unfinished direct prerequisites supplied by F3. It does not replace raw status or lock authorized manual updates. A COMPLETED task is not unfinished Blocked work. Completion-before-prerequisite and transition exceptions remain OD-08; the current protocol spelling does not imply customer approval of all transitions.

Delete rule: the module never silently removes dependency or note references. It asks the reference ports for counts. The policy may reject deletion or define a coordinated cleanup; that choice is Open Decision OD-06. Leader-only deletion follows FR-06; ordinary members do not delete tasks.

6 5 Progress Notes

Element	Description
Purpose	Record short task notes that explain partial progress beyond task-count completion, and return them only to project members.

Element	Description
Main responsibilities	Validate note text and task/project membership; save a note linked to one task; return NoteView; preserve note text on failure.
Inputs	actorId, projectId, taskId, note text.; actorId is UserId; expectedRevision for each existing-project mutation.
Outputs	NoteView (noteId, taskId, authorMemberId, text; optional createdAt); NoteAdded fact. The facade wraps successful writes in MutationResult<T> with revision and SavedChange.
Required information	noteId, taskId, projectId, authorMemberId, text; optional createdAt. Multiple-note history is not assumed.
Dependencies	MG-M0 for permission/transaction; MG-M0 supplies project-membership decisions and MG-M3 task identity; persistence port. It does not change task status or progress counts.
URS requirements	FR-05; NFR-04 (notes private to members); NFR-05.
Error handling	Apply shared safe error mapping. Failure leaves source/revision unchanged and retains caller input; return no private payload. Blank text or length violation: field error, no save. Task not found/not in project: reject. Non-member: deny. Failed save: preserve entered text and previous notes.

Subfunction	Responsibility	Input → output	Main errors
Add Short Task Note	Save a short note against a task; do not modify status/completion.	NoteDraft → NoteView	Blank/too long; task not found; not permitted; save failure
View Task Notes	Return notes for one task to project members only.	projectId + taskId -> authorized NoteView[] with source revision and availability; unreadable/inaccessible data returns PublicError, not a partial list.	Not a member; task not found

Open Decision OD-07: note length, whether multiple notes are retained, whether notes may be edited or deleted, and who may edit another member's note are not fixed by the URS. The design supports one short note per add operation and does not assume a history model.

7 Module Interfaces

7 1 Contract Conventions

The following are conceptual contracts, not API signatures or a wire format. Every existing-project mutation supplies actorId (UserId), projectId and expectedRevision. MG-M0 realizes shared coordination: authorize through MG-M2, read one coherent snapshot, validate, compare revision at the protected commit, save atomically, increment revision once, then publish SavedChange. Project creation initializes revision=1 and has no expectedRevision. Every success is MutationResult<T>; every failure is PublicError with no private saved-record payload.

- Identifiers are opaque; no database key type is prescribed.
- A task assignee is either a project member or explicitly unassigned (null).

- No operation returns a partially saved record; failure means the prior valid state remains.
- Change facts are published after commit, not before; consumers must not rely on a fact for a failed save.
- The response does not expose another project's data; denied access returns no project payload.

7 2 Operation Contracts

Operation (owner)	Request (conceptual)	Success output	Failure outcomes
createProject (MG-M0 → MG-M1, MG-M2)	actorId; name; description; plannedStartDate; projectDeadline; internalTargets; optional initial members.	ProjectView; creator recorded as Leader; MembershipView for the creator. Wrapped in MutationResult<T> with committed revision and SavedChange. Initial revision=1.	INVALID_INPUT, PERMISSION_DENIED, SAVE_FAILED
updateProject (MG-M0 → MG-M1)	actorId; projectId; patch of name/description/dates/internalTargets.; expectedRevision.	Updated ProjectView; date-change fact when dates change. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_INPUT, PERMISSION_DENIED, REVISION_CONFLICT, SAVE_FAILED
manageMembership (MG-M0 → MG-M2, MG-M3)	actorId; projectId; action (add / remove / leave / changeRole); userId; role if applicable.; expectedRevision. Use memberId for existing-membership actions; userId only identifies the person being added.	MembershipView; on departure, DepartureResult with affectedTaskIds and tasks now unassigned. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_OR_INACCESSIBLE_PROJECT, PERMISSION_DENIED, LEADER_AGREEMENT_REQUIRED, INVALID_INPUT, REVISION_CONFLICT, SAVE_FAILED
createTask (MG-M0 → MG-M3, MG-M2)	actorId; projectId; name; description; assigneeId?; durationDays; taskDeadline.; expectedRevision.	TaskView with status NOT_STARTED and assignee/unassigned; TaskChanged fact. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_INPUT, PERMISSION_DENIED, REVISION_CONFLICT, SAVE_FAILED

Operation (owner)	Request (conceptual)	Success output	Failure outcomes
updateTask (MG-M0 → MG-M3, MG-M2)	actorId; projectId; taskId; patch of name/description/durationDays/taskDeadline/assigneeId.; expectedRevision.	Updated TaskView; TaskChanged fact; restricted changes return a permission decision. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_OR_INACCESSIBLE_TASK, INVALID_INPUT, PERMISSION_DENIED, LEADER_AGREEMENT_REQUIRED, REVISION_CONFLICT, SAVE_FAILED
updateTaskStatus (MG-M0 → MG-M3, MG-M2)	actorId; projectId; taskId; newStatus ∈ {NOT_STARTED, IN_PROGRESS, COMPLETED}.; expectedRevision.	Updated TaskView; TaskStatusChanged fact; progress/blocked views refresh downstream. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_OR_INACCESSIBLE_TASK, INVALID_STATUS, PERMISSION_DENIED, REVISION_CONFLICT, SAVE_FAILED
addProgressNote (MG-M0 → MG-M4, MG-M2, MG-M3)	actorId; projectId; taskId; text.; expectedRevision.	NoteView; NoteAdded fact; task status and completion count unchanged. Wrapped in MutationResult<T> with committed revision and SavedChange.	INVALID_OR_INACCESSIBLE_TASK, INVALID_INPUT, PERMISSION_DENIED, REVISION_CONFLICT, SAVE_FAILED

createProject creates the project shell and the creator's Leader membership in one transaction. The URS permits a member list at creation, but invitation delivery/acceptance remains open; the operation may accept initial member identities only

within the agreed policy. updateProject is leader-authorised for project dates; ordinary-member edit rights for descriptive fields are Open Decision OD-09.

manageMembership retains one action discriminator for add/remove/leave/changeRole. Active membership and project-specific roles belong to MG-M2; MG-M3 identifies and clears departing-member assignments through MG-M0. The coordinated result preserves tasks, notes and dependency links. Initial NOT_STARTED is the current design proposal for createTask, not an independently approved transition requirement. updateTaskStatus stays separate because it has distinct permission and progress-refresh effects.

7 3 Information Needed by Other Functional Modules

Consumer	Receives from F1/F2	When / trigger	Deliberately not provided
F3 Dependency Management	projectId, taskId, task name, durationDays, current status, task-created/updated/deleted facts, assignee/unassigned state.	SavedChange after valid task/source mutation. F3 owns edges; F5 alone owns blocked reasons.	F1/F2 does not store dependencies or decide whether a task is blocked.
F4 Schedule Analysis	projectId, plannedStartDate, taskId, durationDays, revision and committed SavedChange facts. F1 also supplies projectDeadline for downstream date comparisons; it does not constrain CPM.	After a valid task/duration/date change; F4 owns calculation and critical activities.	Status does not shorten duration; F1/F2 does not calculate schedule or critical paths.
F5 Progress Tracking	taskId, projectId, name, assigneeId (MemberId|null), status, taskDeadline and revision; optional note text through an authorized read.	After task/status/membership changes; F5 owns counts and dashboard presentation.	F5 derives Blocked from valid F3 links and F2 raw status. Notes never alter completion counts; F5 does not write links or perform CPM.
F6 Schedule Risk Monitoring	TaskSnapshot status, taskDeadline, assigneeId; ProjectSnapshot projectDeadline and revision; committed SavedChange facts.	After saved source/date/status changes or asOfDate refresh; F6 combines F5 facts and current available F4 ScheduleResultView.	F1/F2 does not compose warnings. F6 uses F5 facts and current available F4 results, without performing CPM or inventing a finish date.

7 4 Conceptual Error Codes

Code	Meaning	Data effect
INVALID_INPUT	A required field is missing or malformed; field errors identify the field.	No write.
INVALID_STATUS	Requested status is not one of the three stored values (for example, Blocked).	No status change.
INVALID_OR_INACCESSIBLE_PROJECT	Unknown or inaccessible project reference, including nonmember access. Public message: Invalid or inaccessible project. Internal diagnostics remain private.	No private payload or write; no existence disclosure.

Code	Meaning	Data effect
PERMISSION_DENIED	Actor is a member but lacks the required edit right.	No write; prior values remain.
LEADER_AGREEMENT_REQUIRED	Ordinary member requests a restricted action; the request is not executed by this call.	No write; the facade may offer the leader path; approval workflow is open.
INVALID_OR_INACCESSIBLE_TASK	Unknown or inaccessible task reference. Public message: Invalid or inaccessible task. Accessible-context field errors use INVALID_INPUT.	No private payload or write.

Code	Meaning	Data effect
INVALID_INPUT	Task assignee is not a member of the same project.	No assignment change.
REFERENCE_POLICY_UNRESOLVED	Deletion policy blocks a task with dependency/note references.	No deletion; references remain intact.
SAVE_FAILED	Persistence/transaction failure; valid prior data is preserved.	Rollback; input retained by caller.

8 Design Rules and Validation

8 1 Required Design Rules

#	Rule	How the design enforces it
1	User roles are project-specific.	Role is stored on MG-M2 ProjectMembership; there is no global role field. A user may be Leader in one project and Member in another.
2	Only project members can view private project information.	MG-M0 asks MG-M2 for membership before any read/write; read results are project-scoped. Non-members receive INVALID_OR_INACCESSIBLE_PROJECT without project data.
3	Project Leader has additional management permissions.	MG-M2 returns leader rights for deletion, assignments, project dates and roles. Project Management edits dates; MG-M3 allows task deletion only for a leader.
4	Members can create tasks and perform permitted updates.	MG-M3 createTask is available to members; status and note operations use the permission decision. Restricted updates are not silently allowed.
5	Restricted edits require the agreed permission policy.	MG-M2 returns ALLOW / DENY / LEADER_AGREEMENT_REQUIRED. No approval workflow is invented; the leader-agreement mechanism is OD-05.
6	Removing a member must not delete their tasks.	Departure only removes the membership and returns affected task IDs. MG-M3 clears assignees; no task delete is invoked.
7	Tasks assigned to a departing member become Unassigned.	MG-M0 coordinates MG-M2 departure + MG-M3 unassignment in one transaction. Task records, dependencies and notes remain.
8	Invalid input or failed saves must not destroy valid existing data.	MG-M0 validates before writing and owns rollback; every operation returns a safe failure envelope; preserves prior source and revision. No partial writes.
9	Blocked is a derived warning, not a stored mutually exclusive status.	MG-M3 accepts only NOT_STARTED, IN_PROGRESS, COMPLETED. It rejects Blocked; F5 derive the warning for unfinished tasks and exclude COMPLETED tasks. Blocked never locks a status update.

Additional invariants: a task assignee must belong to the same project; a task cannot be assigned across projects; note addition never changes completion percentage; changing project dates does not silently change task durations or statuses; and no module writes another module's record directly.

8 2 Proposed Validation Scenarios

These are proposed scenarios for later review and testing. They have not been executed; no result in this section is a test result.

ID	Scenario	Expected result
T01	Create a valid project with name, description, planned start, submission deadline, internal targets and the creator as Leader.	Project and Leader membership are saved atomically; creator reopens the project; a non-member cannot read it.

ID	Scenario	Expected result
T02	An Ordinary Member attempts to change the project submission deadline, delete a task or reassign another member's task.	Request returns LEADER_AGREEMENT_REQUIRED or PERMISSION_DENIED; no field changes; prior project/task values remain; explanation is readable.
T03	A project member creates a task with a valid name, positive duration, deadline and a same-project assignee.	Task is saved with status NOT_STARTED; assignee is the selected member; task appears in the project; invalid duration or cross-project assignee is rejected without a partial record.

ID	Scenario	Expected result
T04	Authorize a raw-status update for an unfinished task with an incomplete direct prerequisite. Separately propose COMPLETED before prerequisites.	Blocked is not a hard lock on permitted updates. Apply the agreed transition policy; if COMPLETED is accepted, F5 excludes it from unfinished blockers. Completion-before-prerequisite policy remains OD-08, so this is not an unconditional approval/test result.
T05	Remove a project member who owns two tasks; the tasks also have dependencies and notes.	Active membership ends; both tasks remain; assignee becomes Unassigned; dependencies and notes remain; affectedTaskIds is returned; no task delete occurs.
T06	Simulate a persistence failure during a project/member/task save after validation.	Transaction fails atomically; prior valid source/revision remain unchanged; caller-entered values retained; generic safe SAVE_FAILED explanation, no private prior-record payload.
T07	The same user is Leader in project A and Ordinary Member in project B; attempt a leader-only action in each.	Allowed in A; denied or requires leader agreement in B. No global role leaks between projects.
T08	Add a note to a task, then verify completion counts; access the note as a non-member.	Note is saved and readable by members; completion percentage is unchanged; non-member receives INVALID_OR_INACCESSIBLE_PROJECT and no note text.

8 3 Review Checklist

- Every write has a membership check and a separate edit-permission decision; visibility is not treated as edit rights.
- Every multi-record operation has one transaction boundary: project + leader membership, departure + unassignment, task deletion + reference policy.
- No task is deleted when a member leaves; no task is reassigned to a non-member; no task is silently assigned across projects.
- Blocked is absent from stored status and from the Task table; F5 own the warning.
- A failed save leaves prior data unchanged and returns field errors/input retention information.
- No approval workflow, invitation delivery, notification, export, undo, audit history or priority field has been invented.
- Every open decision is labelled and has an owner/next step in Section 9.

9 Open Decisions

The following decisions are not fixed by the URS or the requirements analysis. The design preserves them as policy inputs or explicit errors; it does not quietly choose a behaviour that the customer has not agreed.

ID	Decision still needed	Affected	Interim design treatment
OD-01	Invitation delivery/acceptance, who may invite, and what makes an invitation valid.	MG-M2; createProject/manageMembership	Membership records can be created, but no delivery/acceptance channel is designed. Add-member and invite are not treated as identical.
OD-02	Leader departure and succession; whether a project must always retain at least one Leader.	MG-M2	Role changes are supported; last-leader rule is a policy result (PERMISSION_DENIED). No automatic succession is invented.

ID	Decision still needed	Affected	Interim design treatment
OD-03	Exact required project fields, date-boundary rules and internal-target representation.	MG-M1	Fields and dates are validated through a policy object; no fixed required/optional list is hard-coded.
OD-04	Task field requirements, self-assignment rights and whether ordinary members may edit task deadlines.	MG-M3; MG-M2	Create/update accept the candidate fields; assignment/deadline changes use the permission decision. Final rights remain open.
OD-05	Whether leader agreement is an approval workflow or the leader simply performs the edit.	MG-M2; MG-M0	Only a decision value LEADER_AGREEMENT_REQUIRED is returned. No approval steps, notifications or voting are designed.
OD-06	Referenced task deletion: reject, or later agree atomic coordinated F3/note cleanup. Active dangling references are never an acceptable outcome.	MG-M3; F3; MG-M4	Until agreed, referenced deletion returns REFERENCE_POLICY_UNRESOLVED without changing source or revision. F1/F2 never writes F3 links directly.
OD-07	Note length, multiple-note history, edit/delete rights and whether notes can be edited by leaders.	MG-M4	One short note per add operation is supported; no history/edit model is assumed.

ID	Decision still needed	Affected	Interim design treatment
OD-08	Final status vocabulary, early completion with unfinished prerequisites, and empty/zero-task behaviour.	MG-M3; F5	ROUND2 TaskStatus uses NOT_STARTED / IN_PROGRESS / COMPLETED; UI labels and early-completion/transition exceptions still need confirmation. Empty progress is NO_TASKS with percentage=null.
OD-09	Whether Ordinary Members may edit project name/description (as opposed to dates).	MG-M1; MG-M2	Project dates are leader-only; descriptive-field edit rights are checked by policy rather than assumed.

Team/customer review should settle OD-01, OD-02, OD-05 and OD-06 because they affect access and atomic preservation. OD-03/04/07/08/09 remain open validation and transition decisions. Cancellation is not a new status or supported operation; it requires scope agreement.

Appendix A Traceability and Glossary

A 1 Requirement Traceability

Requirement	Primary module(s)	Design section	Proposed tests
FR-01 Create Project	MG-M0, MG-M1, MG-M2	3.1, 6.1, 6.2, 7.2	T01, T06
FR-02 Manage Members	MG-M0, MG-M2, MG-M3	3.1, 6.3, 7.2, 8.1	T05, T07
FR-03 Create/Edit Task	MG-M0, MG-M2, MG-M3	3.1, 6.4, 7.2	T03, T08
FR-04 Update Status	MG-M0, MG-M3	3.1, 6.4, 7.2, 8.1	T04
FR-05 Progress Notes	MG-M0, MG-M4	3.1, 6.5, 7.2	T08
FR-06 Permission Control	MG-M0, MG-M2	3.1, 6.3, 8.1	T02, T07
NFR-01 Usability	All	3.2, 6.x, 7.4	T02, T05, T06
NFR-04 Security and Access Control	MG-M0, MG-M2	3.2, 6.3, 8.1	T01, T07, T08
NFR-05 Reliability	MG-M0, all writers	3.2, 6.1, 8.1	T06

A 2 Glossary

Term	Meaning in this design
Project-specific role	Leader or Ordinary Member attached to one ProjectMembership; the same user may hold different roles in different projects.
Management Facade	The root control module that checks permission, sequences module calls and owns the transaction.
Stored status	TaskStatus = NOT_STARTED | IN_PROGRESS | COMPLETED. Display labels are presentation mappings; BLOCKED is not stored.
Derived warning	F5 derives Blocked for unfinished tasks with incomplete direct prerequisites, using valid F3 links and F2 status; no duplicate F3 business judgment.
Open Decision	A decision not fixed by the URS; the design preserves it as policy input or an explicit error instead of inventing behaviour.

A 3 Design Artefacts

File	Purpose
01_Project_Task_Module_Design_ROUND2.docx	This editable design document.
01_Project_Task_Module_Design_ROUND2.pdf	Read-only review copy.
01_Project_Task_Hierarchy_ROUND2.drawio	Editable module hierarchy source (draw.io).
01_Project_Task_Hierarchy_ROUND2.png	Rendered hierarchy used in Figure 1.

No application code, database schema, API framework or deployment configuration was created or modified. The original 14-page received PDF remains unchanged. This editable report was reconstructed from its text/tables because its advertised DOCX and draw.io sources were not available, then revised against the shared Round 2 protocol. Neither the original author nor the team/customer is claimed to have approved these edits. No software tests, commit, push or publication occurred in this round.

Appendix B Round 2 shared contracts

This appendix makes the source-record boundary consistent with the other ROUND2 reports. It is a proposed design protocol, not executed code or customer approval. The original source structure, nine decisions and eight planned validation scenarios are retained; affected text and tables are corrected throughout, rather than only overridden here.

B 1 Canonical source dictionary

Record or field	Shared meaning
ProjectSnapshot	projectId, name, plannedStartDate, projectDeadline, members, tasks, dependencyView, revision. Date fields are LocalDate or null; internal-target representation remains OD-03.
MembershipRecord	memberId, projectId, userId, project-specific role. actorId/viewerId is UserId; assigneeId is MemberId or null. Ended active access does not destroy historical note-author identifiers.
TaskSnapshot	taskId, projectId, name, optional retained description, assigneeId, durationDays, taskDeadline, status. durationDays must be finite/positive when present; requiredness and precision remain OD-04. Preserve canonical input precision for downstream NumericPolicy; F4 analysis validity is separate from F2 save acceptance.
TaskStatus	NOT_STARTED | IN_PROGRESS | COMPLETED. TaskDisplayView carries rawStatus and nullable isBlocked; Blocked is not a fourth stored or mutually exclusive counting value.
ProgressNote	noteId, taskId, authorMemberId, text; optional technical timestamp does not establish retained note history. Length/history/edit rights remain OD-07.
Version fields	revision on source; expectedRevision on mutation input; sourceRevision on derived views. These refer to one project counter, not three independent counters.

B 2 Coherent read and write protocol

SH-COORD is a shared responsibility realized through MG-M0 and the other facades, not a service. It obtains one authorized coherent snapshot from F1/F2 source reads and the F3 relationship read port. One MG-M2 policy governs access; no facade invents a rival permission policy. Creation initializes revision=1. Each successful atomic source change, including status, membership, note, label and date changes, advances revision once. Every existing-project mutation supplies expectedRevision; clients never invent the committed value.

Authorize, read current snapshot, validate, and check expectedRevision inside the protected commit decision. Save all related source records and next revision atomically, then return MutationResult<T> {ok, projectId, revision, value, SavedChange}. SavedChange carries projectId, revision, changeKind, affectedTaskIds and changedFields. A direct coordinator call is sufficient; no broker is implied. Rejected, conflicting or save-failed requests publish no successful change and do not increment revision. Return PublicError and retain entered input without returning private saved records.

F3 dependency cycle validation and expectedRevision comparison use the same protected commit decision. Two clients adding reverse links from one revision cannot both commit a cycle: the second conflicts or revalidates against the newly saved graph. Locks or compare-and-save storage mechanics remain detailed-design choices.

readProjectSnapshot(projectId, actorId) is the conceptual authorized source-read port used by coordination. It returns a ProjectSnapshot with coherent F1/F2 source records and revision, or a safe PublicError. Reauthorize each read; if F1/F2 records cannot be read safely and coherently, return SOURCE_UNAVAILABLE without a partial private snapshot. A dependency-only capability failure is represented by dependencyView and does not discard safely readable current task/member facts. This names an existing source-read responsibility, not a new application feature.

ProjectView, TaskView, MembershipView and NoteView refer to the same coherent source revision. Successful authorized source reads identify projectId, revision and availability; inaccessible or unreadable core data returns PublicError without a private or partial payload. Permission decisions use the current authorized snapshot. SH-COORD establishes that projectId/revision context for task, membership and complete dependency reads; DependencyResultView.sourceRevision must equal the ProjectSnapshot.revision it supports. After a note or member-role change, reauthorize and prove the complete validated edge set and current task universe still agree, or re-read them coherently, before publishing the view at the current sourceRevision. Relevant task/edge changes require fresh validation. Failed coherence proof gives DependencyResultView UNAVAILABLE/STALE_RESULT with edges=null; missing/unreadable complete input gives UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE. F4 maps either untrusted dependency condition to UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE for nonempty tasks. No dependencyRevision or scheduleRevision

counter is introduced. Version comparison, cycle validation and atomic save remain one protected commit decision; storage locks/transactions are design constraints, not implemented mechanisms.

B 3 Explicit deleteTask contract

Item	Proposed contract
Ownership	MG-M0 coordinates; MG-M3 owns task lifecycle; MG-M4 notes and F3 dependency references retain separate ownership. This makes an already-described Delete Task operation explicit, rather than adding an unsupported feature.
Input	actorId (UserId), projectId, taskId, expectedRevision; authorized deletion policy.
Success	MutationResult<TaskDeleted> with next revision and SavedChange only after an atomic valid commit.
Unresolved references	Consult F3/note read ports; if their cleanup policy is not agreed, return REFERENCE_POLICY_UNRESOLVED. No task, link, note or revision changes.
Later agreed cleanup	A coordinator may atomically request owning modules to clean references with task deletion. F1/F2 never directly edits F3 records. No active dangling reference may remain.
Other errors	INVALID_OR_INACCESSIBLE_PROJECT/TASK; PERMISSION_DENIED or LEADER_AGREEMENT_REQUIRED only in accessible context; INVALID_INPUT, REVISION_CONFLICT, SAVE_FAILED.
Cancellation	No CANCELLED state or cancellation operation is introduced. Scope and transition rules require a separate decision.

B 4 Refresh and optional analysis

All derived results carry availability and sourceRevision. F5/F6 use a schedule only when its published sourceRevision equals the coherent snapshot revision. Task creation/deletion, dependency, duration and plannedStartDate changes invalidate related old schedule results. Status changes refresh progress/Blocked; taskDeadline/projectDeadline changes refresh comparisons; membership/assignee changes refresh access/assignment views. asOfDate is a read input and does not increment revision.

The coordinator may reuse relative timings only after proving task IDs, graph and durations unchanged, remapping absolute finish if plannedStartDate changed, refreshing labels/date comparisons and publishing current sourceRevision. If this proof cannot be made, return UNAVAILABLE with STALE_RESULT rather than stale current critical flags or dates. No independent schedule-input version counter is added.

Consumer	F1/F2 supply and boundary
F3	Authorized task identities and lifecycle/source facts. F3 validates/saves/removes and supplies links only; F5 alone derives Blocked.
F4	Current task durations/IDs, valid F3 links, plannedStartDate, revision; F4 alone produces ScheduleResultView and estimatedProjectFinish. F1/F2 never performs CPM or shortens duration from status.
F5	TaskSnapshot status, taskDeadline, assignee MemberId and coherent revision. F5 derives raw counts, completion, assigned tasks, basic overdue facts and Blocked from valid direct F3 links.
F6	Task/project dates and status plus F5 facts and available current F4 ScheduleResultView. F6 composes only retained FR-15 types and performs no schedule calculation.

B 5 Availability and formal scope

ScheduleAvailability = VALID | NO_TASKS | NOT_ENABLED | UNAVAILABLE. VALID means nonempty tasks and a reliable current schedule. NO_TASKS means F4 enabled with no tasks, with null duration/criticalTaskIds/estimatedProjectFinish, not zero-day delivery. F4 off is NOT_ENABLED with reasonCode=FEATURE_NOT_ENABLED. UNAVAILABLE means reliable current analysis cannot be supplied; missing facts remain unknown, not empty/no-risk assertions.

ScheduleResultView UNAVAILABLE reasonCode is INVALID_INPUT, CALCULATION_FAILED, STALE_RESULT or DEPENDENCY_INPUT_UNAVAILABLE. DependencyResultView UNAVAILABLE reasonCode is DEPENDENCY_INPUT_UNAVAILABLE (missing/unreadable complete input), INVALID_INPUT (invalid records/graph), or

STALE_RESULT (current coherence unproved); never CALCULATION_FAILED. Each type uses FEATURE_NOT_ENABLED only for its own NOT_ENABLED state; VALID/NO_TASKS have no failure reasonCode. BlockedResult uses the applicable dependency reason, not an F4 calculation reason. ProgressResult/OverdueResult do not inherit F4 reasons. Enabled ScheduleWarningResult uses evaluationByType reasons only for that type's unavailable inputs; independent overdue evaluation does not inherit an F4 failure. F6 off is NOT_ENABLED/FEATURE_NOT_ENABLED with null warning payload; date/finish explanations remain separate.

Evaluate F4 gates in order: off -> NOT_ENABLED/FEATURE_NOT_ENABLED; enabled and empty -> NO_TASKS; enabled with at least one task but no complete, valid, same-revision dependency view -> UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE. Only then calculate a current schedule from valid inputs. For DependencyResultView, F3 off -> NOT_ENABLED/FEATURE_NOT_ENABLED with edges=null; enabled empty task universe -> NO_TASKS with edges=[]; enabled nonempty tasks with complete valid edges -> VALID, including a verified edges=[] set. edges=null never satisfies the dependency gate. No dependency-free override is approved.

FR-09 Should; FR-10/11/12 Could; FR-13/14 Must; FR-15 Could. FR-13 basic Dashboard maps to MON-M0 / MON-M1 / MON-M2 / MON-M4 / MON-M5 + UI for coordination, raw status, completion, personal assignments, basic overdue facts and presentation. FR-13 Blocked subview maps separately to MON-M3; dependent on Should Have FR-09 / F3; release decision D-01 remains pending. If F3 is off/unavailable, label Blocked capability NOT_ENABLED/UNAVAILABLE with unknown flags/reasons, and retain basic Dashboard functions. Basic FR-13 progress/status/assignment/overdue display continues without F4/F6. With safe current task facts and valid taskDeadline/asOfDate, MON-M5 overdue facts remain evaluable during F4 failure. If F6 is enabled, OVERDUE_TASK may be evaluated independently; BLOCKED_CRITICAL_TASK and PROJECT_DEADLINE_CONFLICT remain unevaluated when their required current criticalTaskIds or estimatedProjectFinish is unavailable. Unevaluated is unknown, not no risk. F6 off does not make FR-15 warnings available; FR-15 remains Could. D-01, date decision D-04 and the other existing D-02 to D-10 / OD-01 to OD-09 decisions remain pending; no formal priority, permission or acceptance decision is silently changed. Empty progress is NO_TASKS with percentage=null, never division by zero.

Weekly precision and diagnostic recommendation (D-03/D-07 PENDING): accept finite positive analysis values, including decimals, preserve canonical digits and avoid intermediate rounding. F4 owns the centralized NumericPolicy and certified criticalTaskIds; F1/F2 does not classify display-rounded float. Whether omitted duration may be saved remains a core task-policy decision; optional F4 failure never automatically rejects a policy-valid F2 save. Authorized UNAVAILABLE/INVALID_INPUT may carry DurationInputDiagnostic taskId, field=durationDays, issue=MISSING/NON_NUMERIC/NON_FINITE/NON_POSITIVE and message. All schedule timing/critical/finish facts remain null; diagnostics are not partial results. Denied/unknown references receive only the existing generic PublicError.

B 6 Access and candidate date semantics

Unknown/nonmember/inaccessible project references return INVALID_OR_INACCESSIBLE_PROJECT with “Invalid or inaccessible project.” Task references return INVALID_OR_INACCESSIBLE_TASK with “Invalid or inaccessible task.” Do not disclose existence, private names/titles, memberships or previous saved values. Accessible contexts may receive detailed field/relationship errors. PERMISSION_DENIED and LEADER_AGREEMENT_REQUIRED follow established accessible membership context; no approval workflow is invented.

D-04 candidate: calendar days; Day0=plannedStartDate; estimatedProjectFinish=plannedStartDate+ceil(T), using canonical unrounded F4 duration T only when that mapping is selected for a conditional scenario or approved. Integer T uses start+T. Unfinished overdue iff known taskDeadline and asOfDate>taskDeadline; project conflict iff current known finish>projectDeadline. Equality triggers neither. Missing dates/mapping leave comparisons unevaluated. Project calendar/timezone and deadline inclusion remain PENDING; chat timezone grants no approval. F1/F2 supplies dates and never calculates finish.

B 7 Source preservation and next review

Source received: 01_Project_Task_Module_Design(1).pdf, 14 pages, SHA-256 5ed66aafb571c4ac8e5aa49082bac2d654bd90d5d79474c7f3240042cba4161d. All original substantive sections 1-9 and Appendix A are reconstructed as editable paragraphs/tables, with disclosed targeted edits. The original file is kept unchanged in Sources/. The revised source basis is the verified 24-page URS_REVISED and 36-page Requirements Analysis REVISED; artifact identity is not customer approval. Detailed change mapping is saved in the Round 2 evidence index.

Next: original owner and team review the integration edits, decide OD-01-09 and shared priority/date/availability questions, finalize detailed contracts, and only then plan implementation and software tests. The original T01-T08 and integration scenarios are NOT RUN design plans. The current shared plan has27 NOT RUN integration scenarios: IT25 validates current CPM-to-progress/warnings, IT26 precision, IT27 authorized invalid-duration diagnostics and safe core-save/degradation separation. Recommendations do not approve OD-01..09 or D-01..10.